

// **WAYSCONF 2026 · MASTERCLASS · 90 MINUTES**

---

# Build an AI that checks and fixes its own work

Agentic loops in practice.

Szymon Paluch · CTO, Susteen

ROOM-PM · 16 September 2026

// 0:00 – 0:08

---

# The case

# Most AI demos stop here

A prompt goes in. Something impressive comes out. Everyone claps.

Then someone asks a question the demo was not built to survive:

*Does it match the design?*

*Is the contrast legal?*

*Does it work on a phone?*

*Is that the actual copy, or copy it invented?*

no answer

---

# This is a talk about the word *no*

A loop is only worth building if something in it can refuse  
the work without asking a language model for an opinion.

---

# What you will have at 16:25

A landing page for a music festival that does not exist, built from a Figma file by an agent you instructed, checked by a program that agent wrote and is not allowed to change, fixed when it failed, and deployed to a URL you can send to someone.

```
exit 0
```

## YOU LEAVE WITH

A URL, live on the internet.

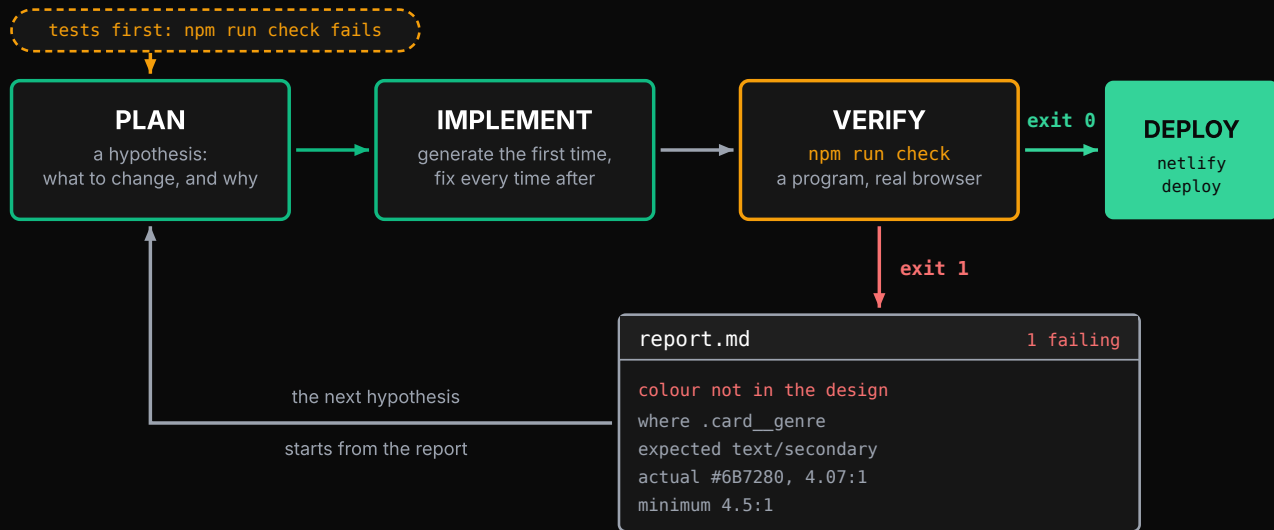
A folder: the page, and the checker that judged it.

Eight prompts that work on Monday, on something that is not a festival.

# The loop

## The loop

The check is written first and fails. Then plan, implement, verify, until it passes.



The way back carries a file. Nothing on it asks the model whether the work is good.

# Setup

**An agent running in an empty folder**

# You need two things

## THE DESIGN

`turbine.fig`, downloaded from the page — the file your agent reads. The same page opens the design in Figma, to look at.

## THE PROMPTS

[waysconf.szymonpaluch.com](https://waysconf.szymonpaluch.com)

Eight of them, with a copy button on each.

## AND YOUR EMPTY REPOSITORY

```
cd turbine      # made at home
claude          # or: codex
```



It is **empty**, and there is no code to read.

Agent up? Paste **prompt 01** now.



# The terminal, in one slide

```
pwd          # where am I
ls           # what is here
cd turbine   # go in there
claude       # or: codex
```



**Ctrl+C** stops whatever is running.

It is a text field that runs programs.

It gives you no feedback while it works, which reads as "broken" and is almost always "busy".

When a command finishes you get the prompt back. When you do not get the prompt back, it is still going.

Full version: the Before tab, for your operating system

# Every step runs the same way

|              | WHAT YOU DO                                                      | CLAUDE CODE                                      | CODEx                   |
|--------------|------------------------------------------------------------------|--------------------------------------------------|-------------------------|
| 1. Plan      | Switch to plan mode, paste the prompt, read the plan, approve it | <code>Shift+Tab</code> until <i>plan mode on</i> | <code>/plan</code>      |
| 2. Implement | The agent builds what the plan says                              |                                                  |                         |
| 3. Commit    | The agent commits the step and pushes it to your repository      | written into the prompt                          | written into the prompt |
| 4. Clear     | Empty the context before the next prompt                         | <code>/clear</code>                              | <code>/clear</code>     |

Prompts 02, 03 and 04 start in plan mode. The next prompt reads its state from files, not from the conversation.

// 0:15 — 0:25

---

# Design in, not screenshots in

# Plan mode, then prompt 02

Put `turbine.fig` in `design/` inside your project. Switch to **plan mode**, paste **prompt 02**, read the plan, approve it.

## WHAT IT BUILDS

A decoder in your project: `npm run design`. It reads the `.fig` and writes the design as JSON in `design/data/`: sections, colours, typography, layout, components, copy, images.

The checker and the page both read those files. Nobody decodes or exports the design again.

## WHAT YOU DO

Approve the plan, then wait ten to fifteen minutes. Let it use the network if it asks.

It writes no page. It stops with a list and its open questions (point 6). Answer them; it commits and pushes. Then `/clear`.

# Two ways to hand over a design

## A SCREENSHOT

The agent looks at pixels and infers.

The orange becomes *an* orange.

The spacing becomes *some* spacing.

It looks roughly right.

token gate fails

## THE FILE ITSELF

Colours arrive as values. Spacing arrives as numbers.

```
TURBINE / Color
accent/sodium    #FF6A1A
text/secondary   #A7AEBB
text/muted       #6B7280
```



Nothing is guessed, so nothing is guessed wrong.

token gate passes

# The file, and what happens to it

## SO YOU HAND OVER THE FILE

|             |                        |
|-------------|------------------------|
| turbine.fig | from the workshop page |
| canvas.fig  | the whole document     |
| images/     | every photo            |



Nothing in this workshop is blocked by a free Figma account.

## AND THE AGENT BUILDS A DECODER

It has never seen a `.fig`. It plans a decoder, writes it, runs `npm run design`, and fixes it until twelve cards carry twelve names. The result stays in the project as JSON, for the checker and for the page.

# The verifier

Written before the page

# Plan mode, then prompt 03

Answer 02's open questions first. Then `/clear`, switch to **plan mode**, paste **prompt 03**.

Your agent plans and writes `npm run check` from `design/data/`, before any page exists, and runs it on the empty project.

it must be red

About eight minutes. A check that passes on an empty page is not checking anything. It commits; then `/clear`.

---



# Tests first

## RED

The check is written from the design data and run on the empty project.

It fails. The report lists everything the page does not have yet.

prompt 03

## GREEN

The page is built against that report, then fixed until the check passes.

prompts 04 and 05

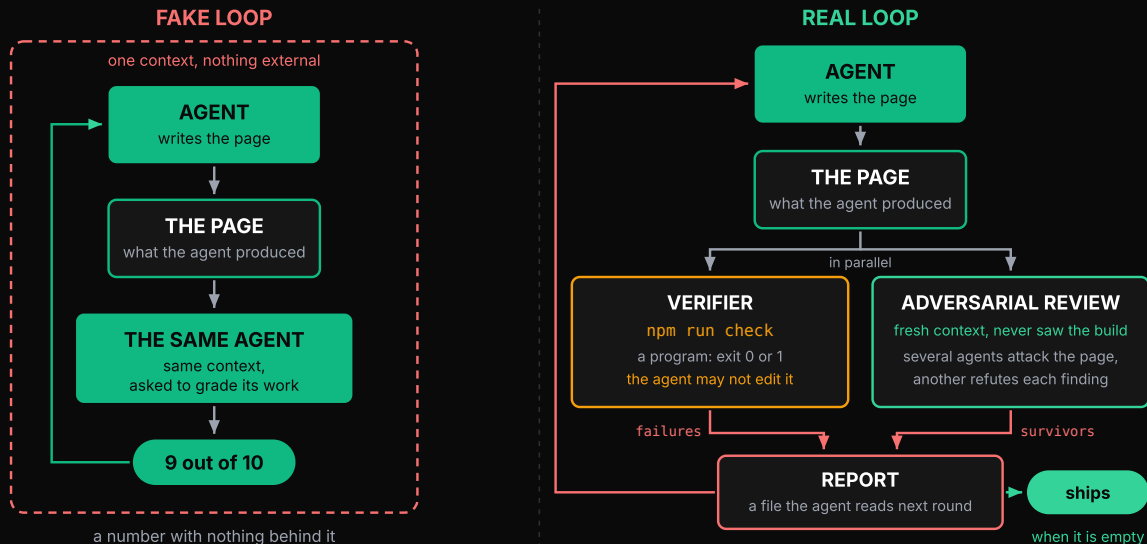
A check written after the page describes the page. A check written first can only describe the design.

---

# A real loop and a fake one

## Fake loop, real loop

An agent can check work. Not the same agent, in the same context, that did the work.



The program decides what can be measured. The fresh-context review looks for what cannot.  
Neither replaces the other, and neither is the agent grading itself.

# Who checks the work

## WHO CHECKS

## GOOD FOR

The same agent, same context

Nothing. It repeats its own reasoning

A program: `npm run check`

Anything with an exact answer: sections, colours, copy, contrast

Fresh-context agents, told to attack

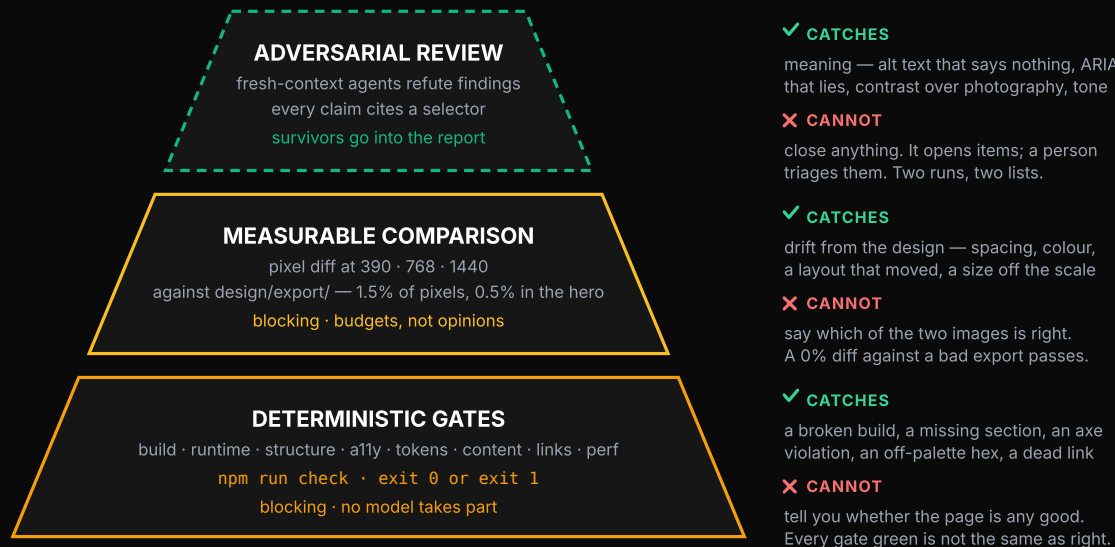
What a program cannot measure: reading order, alt text, text over photos, tone

Run the last two **side by side**. One report. It ships when the report is empty.

# Three tiers of checking

## Three tiers of verification

The bottom tier is the widest, the cheapest and the only kind that can be trusted to say no on its own.



The build gate runs first and alone; the seven browser gates then run together, and Lighthouse last.

Only the bottom two tiers can fail the build. Adversarial review runs alongside npm run check; what survives goes into the report.

Automated accessibility rules reach roughly 30 to 40 per cent of WCAG — a widely cited estimate, not a measurement of this page.

# The reference site: ten gates, no opinions

| GATE                    | WHAT IT DECIDES                                          | CRITERIA     |
|-------------------------|----------------------------------------------------------|--------------|
| Build                   | It compiles, no type errors, one page out                | AC-01...05   |
| Runtime                 | No console errors, no failed requests                    | AC-03...04   |
| Structure               | Sections, headings, landmarks, skip link, ids            | AC-06...14   |
| Accessibility           | axe clean at three widths, focus, keyboard, motion       | AC-15...25   |
| Design tokens           | Every colour and size came from the design               | AC-26...33   |
| Visual fidelity         | Pixels match the design, within a stated budget          | AC-34...37   |
| Content                 | The real copy, all twelve artists, no placeholders       | AC-38...47   |
| Links                   | Nothing points at nothing                                | AC-48...51   |
| Lighthouse              | Performance, a11y, best practices, SEO, and five budgets | AC-52...56   |
| <b>Text over images</b> | <b>The contrast axe refuses to judge</b>                 | <b>AC-61</b> |

# Watch it catch something real

The palette has a trap in it, on purpose.

```
color.text.muted    #6B7280  
color.bg.base       #0A0B0D
```



4.07 : 1

Under the 4.5:1 that body text requires.

```
color.text.secondary #A7AEBB  
color.bg.base        #0A0B0D
```



8.9 : 1

Ask any model for "a muted grey for supporting copy" and watch which one it reaches for.

# The failure report is the interface

```
**3. [check 07] a colour on the page is not in the design**
```

- Where: the supporting line inside each card in the lineup section
- Expected: the design's "text / secondary"
- Actual: the design's "text / muted" – measured 4.07:1 against the page background,  
and body text needs 4.5:1
- Hint: muted is defined for legal and footer text only



No model wrote that. A program measured it and printed it.

The agent's next iteration starts by reading this file.

---

# The rule that holds it all up

The agent may never edit the verifier.

It is in prompt 05, in capitals, and it comes first:

```
1. NEVER change the checker to make a check pass. Not a threshold,  
   not a skipped assertion, not a rule switched off. If you believe  
   a check is wrong, STOP, tell me which one and why.
```





# Here is what actually happened

These prompts, an **empty folder** outside every repository, nothing else.

| PROMPT                             | AGENT       | RESULT                                         |
|------------------------------------|-------------|------------------------------------------------|
| 01 — a project exists              | Codex       | 107 s                                          |
| 02 — reads the <code>.fig</code>   | Claude Code | 16 min — 12/12 names, every colour, all widths |
| 03 — a hero from the design        | Codex       | 41 s                                           |
| 04 — <b>writes its own checker</b> | Codex       | 495 s — 264 lines, tests unasked               |
| 05 — the loop runs                 | Codex       | 46 s                                           |

Unasked, 04 treated axe's *incomplete* results as failures — from one sentence: **"a check that cannot run is a failure, never a skip"**.

# And here is what went wrong

Thirteen incidents while building this. **Nine were failures of the verifier or the harness, not of the page.**

## WHAT

Nine gates ran against **somebody else's website** for a whole run — 427 confident, correctly-formatted failures

**axe passed a blank page.** Zero violations, three breakpoints, green

The loop hung for an hour on a stdin nobody closed

The **evidence harness fabricated** four clean iterations out of a killed run

The gate was green and **the hero was illegible** — five of nine pieces of text below the legal minimum

## WHY IT IS IN THE DECK

A verifier that is confident and wrong is worse than none

A measurement of nothing looks exactly like a measurement of perfection

Third unbounded wait in one project

Absence read as success — fourth time, and the only one that invented a table

axe does not fail text over a photograph. It marks it *incomplete*

# Build the page

# Read the plan before it writes the code

Plan mode is the cheapest correction in the entire loop. A wrong assumption costs one sentence to fix here, and twenty minutes of generated code to fix later.

```
# Claude Code: Shift+Tab until the footer says "plan mode on"  
# Codex: /plan
```



Then read it. Actually read it. The plan is where you find out it misunderstood the brief.

---

# Your turn — 15 minutes

## 1. PLAN MODE, PROMPT 04

It plans the page from `design/data/`, builds all of it, and runs the check once. Still red, with fewer failures.

## 2. REVIEW IT

Beside Figma, one section at a time. Say what is wrong in plain words. Then it commits, and you `/clear`.

Falling behind cannot hurt you.

Stop where you are. Leave whatever is unfinished. I want to move on.

Every prompt stands alone. A half-built page with a working checker teaches more than a finished page with none.

// 0:55 — 1:10 • SPRINT 2 RUNS IN THE BACKGROUND

---

# The loop, and workflows

# Start your loop now, then listen

`/clear`, then paste **prompt 05**. Then leave it alone — I will talk over it, and when it stops we will look at what it did.

```
Run npm run check. If it exits 0, stop and tell me.  
If it does not, read the report it wrote, fix what it names,  
run it again. Repeat until it exits 0.  
  
NEVER change the checker to make a check pass.
```



On Claude Code you can make it structural instead of trusted:

```
/goal npm run check exits 0
```



The session cannot end until that command actually exits 0. Codex has no equivalent — there, the paragraph is the mechanism.

---

# The whole loop is four sentences

|                                |                                                                                                          |
|--------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Check first</b>             | Run the check <i>before</i> you fix anything. An agent asked to improve passing work will find a reason. |
| <b>Hand back the report</b>    | Unchanged. The file a program wrote, not your summary of it.                                             |
| <b>Never touch the checker</b> | The moment the thing being judged can edit the judge, every green result afterwards means nothing.       |
| <b>Stop on a program</b>       | Not when it feels finished. When something exits 0.                                                      |

No script. Nothing to install. You can say this to any agent, today.



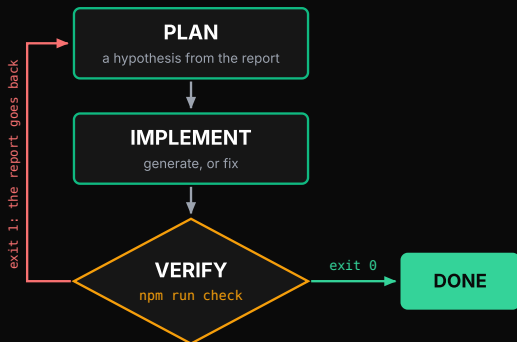
# Two shapes, and neither is a script

## Two shapes

A loop repeats until a condition holds. A workflow runs phases in order, and a phase can branch.

### THE LOOP

repeat until a condition holds



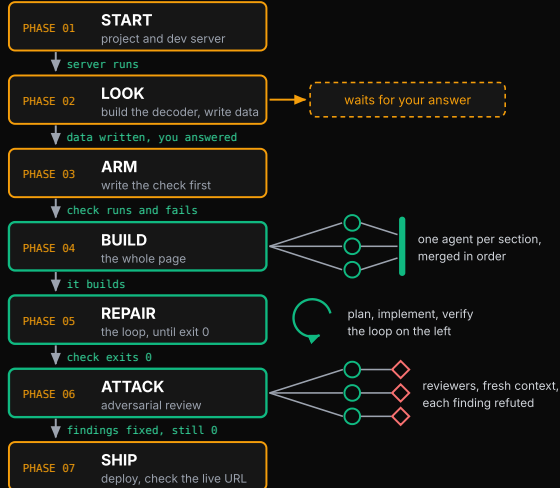
You define the exit condition.

Check written first. Then plan, implement, verify, until exit 0.

```
prompt 05 · /goal npm run check exits 0
```

### THE WORKFLOW

phases in order, a bar between each



You define the phases, what branches inside them, and the bar between.

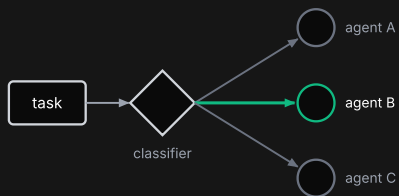
```
prompt 08
```

They nest: phase 05 of the workflow is the loop. Both are written as instructions, not code.

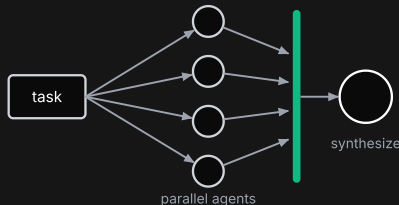
# Six workflow patterns

## Six workflow patterns

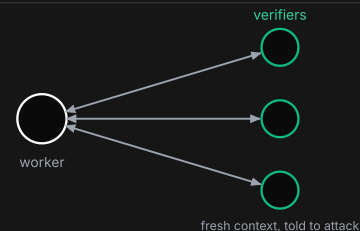
### 1 Classify and act



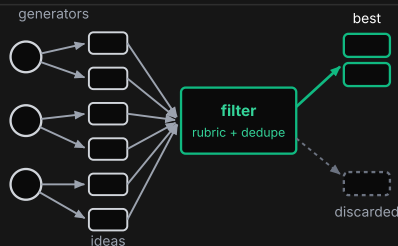
### 2 Fan out and synthesize



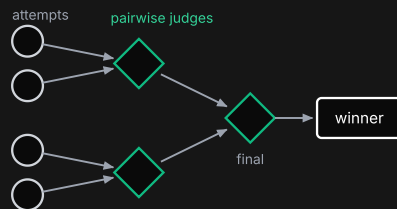
### 3 Adversarial verification



### 4 Generate and filter



### 5 Tournament



### 6 Loop until done



# Why a fresh context beats a long one

## ONE LONG CONVERSATION

Forty minutes of history.

Three abandoned approaches still in context.

The original brief, sixty thousand tokens ago.

Reasoning gets worse as it fills.

## A FRESH CONTEXT EACH PASS

Reads the report — the current failures.

Reads `notes.md` — what was already tried, and what it cost.

Reads the brief.

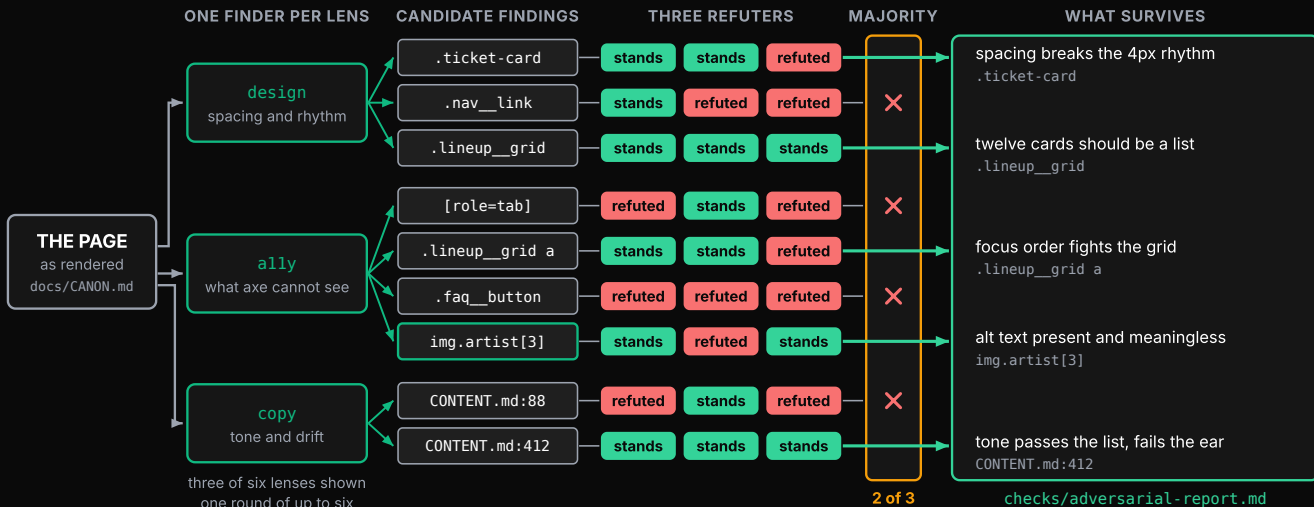
Notes on disk beat memory in context.

---

# When one agent is not enough

## One finder per lens, three refuters per finding

The work fans out, gets argued with, and only the part that survives a majority reaches a person.



### Nothing here runs in lockstep.

Each candidate goes to three refuters whose only instruction is to argue that it is wrong, and reaches the filter when its own panel finishes. A lens that returns two findings does not hold up a lens that returns four.

The filter is arithmetic: two of three must stand. Every claim cites a selector or a file and line, or it is discarded unread.

Nothing on this page sets pass or fail. It opens items for a person to triage.

# Adversarial review

The instruction is not "review this".

The instruction is **"refute this, and default to refuted if you are unsure"**.

```
ultracode. ... inside a phase, spawn as many subagents as the work  
needs and run them in parallel, deciding how many from what you find
```

```
PHASE 07 – ATTACK
```

```
... for every candidate they find, a separate subagent whose only  
job is to argue against it from those three angles.
```



That is **prompt 08**: prompts 01–07 as one message, phase for phase, word for word. Take it home.  
Today, **prompt 06** in a new session is the one-agent version, before you deploy.

---

# Review, then ship

# Break it, then deploy it

`/clear` , then **prompt 06**. An agent that never saw the build tries to break the page. Pick the findings that are real; it fixes them, and the check still passes.

`/clear` , then **prompt 07**, the last step. It builds, deploys to Netlify, and checks that the page the server returns is the page that passed.

It asks you to pick or create a site. Take the URL it prints.

Put your URL on the board. We are going to look at all of them.

---

# The gate was green. The hero was illegible.

Zero axe violations. Three breakpoints. Six page states. Twice, deterministically. Lighthouse accessibility **100**.

Then I measured the hero by hand — screenshot the page with every glyph made transparent, sample the brightest patch behind each line, compute the real ratio.

| TEXT                             | MEASURED |
|----------------------------------|----------|
| "Fourth edition"                 | 2.94:1   |
| "Ambient, techno and modular..." | 3.02:1   |
| "The Powerhouse, Hall E"         | 4.25:1   |
| "Friday 12 – Sunday 14 June"     | 4.41:1   |

## WHY AXE SAID NOTHING

It does not evaluate text over a background image. It does not fail it — it marks the pair **incomplete**, and in a zero-violations gate that is indistinguishable from correct.

Reading the CSS would not have found it either. The background there is a photograph, two scrims and a gradient composited together. No computed style says what colour that is.

So it became AC-61, and there are sixty-one criteria now instead of sixty.



# What this does not do

# Honesty

## THE CLAIM

The a11y gate is green

The pixel diff passes

Adversarial review found nothing

All sixty-one criteria pass

## THE LIMIT

Automated rules reach maybe 30–40% of real WCAG failures

It saw no change. It has no idea whether a change would be better

A review finds what it looks for. It runs beside the program, never instead of it

Nothing here has an opinion about whether the design is good

Green means **no known defect**. It has never meant finished.

# Where loops actually fail

**It edits the test.** The gate goes green and nothing was fixed. Forbid it in writing, and check that the file did not change.

**It oscillates.** Two wrong states, alternating. Usually two criteria that contradict each other. Cap the iterations and read the log.

**It keeps going after it is done.** Run the verifier *first*, every pass.

**It runs out of context mid-task.** Put the state in a file.

**It reports success without running anything.** Only believe the exit code.

**It hangs.** Not failing, just waiting. Every step needs a deadline — this project hung three times: six minutes, seven, and an hour.

---

# Take this to work on Monday

Do not start with a whole landing page. Start with one gate.

1. Pick the check your team already does by hand and resents.
2. Make it a program that exits 0 or 1.
3. Put the failure output somewhere an agent can read.
4. Only then put an agent in front of it.

The loop is the easy part. The oracle is the work.

---

// EVERYTHING FROM TODAY

---

# TURBINE

waysconf.szymonpaluch.com

The eight prompts, in English and Polish · the `.fig` · these slides  
and the repository, with the checker and the evidence: `github.com/hculap/waysconf-2026-agentic-loops`

Questions. And then go and put your URL on the board.